19. HTTP API 서버 구현 분석

개요

Lighthouse의 HTTP API 서버는 Ethereum Beacon API 표준을 구현하여 외부 클라이언트와 검증자들이 beacon node와 상호작용할 수 있도록 하는 핵심 컴포넌트입니다. `beacon_node/http_api` 모듈에서 구현되며, Warp 웹 프레임워크를 기반으로 RESTful API를 제공합니다.

아키텍처 구조

1. 전체 구조

```rust
// beacon_node/http_api/src/lib.rs

pub struct Context<T: BeaconChainTypes> {

pub config: Config,

pub chain: Option<Arc<BeaconChain<T>>>,

pub network_senders: Option<NetworkSenders<T::EthSpec>>,

pub network_globals: Option<Arc<NetworkGlobals<T::EthSpec>>>,

pub beacon_processor_send: Option<BeaconProcessorSend<T::EthSpec>>,

pub sse_logging_components: Option<SSELoggingComponents>,

}
```

Context 구조체는 HTTP 서버가 필요로 하는 모든 의존성을 포함합니다:

- `config` : 서버 설정 (포트, TLS, CORS 등)

- `chain` : BeaconChain 인스턴스에 대한 접근
- `network_senders` : 네트워크 메시지 전송을 위한 채널
- `network_globals` : 네트워크 상태 정보
- `beacon_processor_send` : 비동기 작업 처리를 위한 채널

2. 서버 설정

```rust
#[derive(PartialEq, Debug, Clone, Serialize, Deserialize)]
pub struct Config {
pub enabled: bool,
pub listen_addr: IpAddr,
pub listen_port: u16,
pub allow_origin: Option<String>,
pub tls_config: Option<TlsConfig>,
pub data_dir: PathBuf,
pub sse_capacity_multiplier: usize,
pub enable_beacon_processor: bool,
pub duplicate_block_status_code: StatusCode,
pub target_peers: usize,
}
```

기본 설정값:

- 포트: 5052
- 주소: 127.0.0.1 (로컬호스트)

- TLS: 비활성화
- CORS: 설정 가능

주요 엔드포인트 분석

1. Beacon API 엔드포인트

Genesis 정보

```
// GET /eth/v1/beacon/genesis
let get_beacon_genesis = eth_v1
.and(warp::path("beacon"))
.and(warp::path("genesis"))
.and(warp::path::end())
.and(task_spawner_filter.clone())
.and(chain_filter.clone())
.then(|task_spawner: TaskSpawner<T::EthSpec>, chain: Arc<BeaconChain<T>>| {
task_spawner.blocking_json_task(Priority::P1, move || {
let genesis_data = api_types::GenesisData {
genesis_time: chain.genesis_time,
genesis_validators_root: chain.genesis_validators_root,
genesis_fork_version: chain.spec.genesis_fork_version,
};
Ok(api_types::GenericResponse::from(genesis_data))
})
```

```
});
```

상태 관련 엔드포인트

```
// GET /eth/v1/beacon/states/{state_id}/root
let  get_beacon_state_root = beacon_states_path
.clone()
.and(warp::path("root"))
.and(warp::path::end())
.then(|state_id: StateId, task_spawner: TaskSpawner<T::EthSpec>, chain: Arc<Beac
task_spawner.blocking_json_task(Priority::P1, move || {
let (root, execution_optimistic, finalized) = state_id.root(&chain)?;
Ok(api_types::GenericResponse::from(api_types::RootData::from(root)))
.map(|resp| resp.add_execution_optimistic_finalized(execution_optimistic, finali
})
});
```

2. 검증자 관련 엔드포인트

검증자 정보 조회

```
// GET /eth/v1/beacon/states/{state_id}/validators
pub  fn  get_beacon_state_validators<T: BeaconChainTypes>(
state_id: StateId,
```

```
chain: Arc<BeaconChain<T>>,

query_ids: &Option<Vec<ValidatorId>>,

query_statuses: &Option<Vec<ValidatorStatus>>,

) -> Result<ExecutionOptimisticFinalizedResponse<Vec<ValidatorData>>, warp::Reje

let (data, execution_optimistic, finalized) = state_id

.map_state_and_execution_optimistic_and_finalized(

&chain,

|state, execution_optimistic, finalized| {

let  epoch = state.current_epoch();

let  far_future_epoch = chain.spec.far_future_epoch;

let  ids_filter_set: Option<HashSet<&ValidatorId>> =

query_ids.as_ref().map(HashSet::from_iter);



Ok((

state.validators().iter()

.zip(state.balances().iter())

.enumerate()

.filter(|(index, (validator, _))| {

// ID 필터링 로직

})

.filter_map(|(index, (validator, balance))| {

// 상태 필터링 로직

})
```

```
.collect::<Vec<_>>(),
execution_optimistic,
finalized,
))
},
)?;
// 응답 반환
}
```

검증자 듀티 조회

```
// GET /eth/v1/validator/duties/attester/{epoch}
pub fn attester_duties<T: BeaconChainTypes>(
request_epoch: Epoch,
request_indices: &[u64],
chain: &BeaconChain<T>,
) -> Result<ApiDuties, warp::reject::Rejection> {
let current_epoch = chain.epoch().map_err(warp_utils::reject::unhandled_error)?
if request_epoch == current_epoch || request_epoch == current_epoch + 1 {
cached_attestation_duties(request_epoch, request_indices, chain)
} else if request_epoch > current_epoch + 1 {
Err(warp_utils::reject::custom_bad_request(format!(
"request epoch {} is more than one epoch past the current epoch {}",
```

```
request_epoch, current_epoch
)))
} else {
compute_historic_attester_duties(request_epoch, request_indices, chain)
}
}
```

3. 블록 관련 엔드포인트

블록 발행

```
// POST /eth/v1/beacon/blocks
pub  async  fn  publish_block<T: BeaconChainTypes, B: IntoGossipVerifiedBlock<T>
block_root: Option<Hash256>,
provenanced_block: ProvenancedBlock<T, B>,
chain: Arc<BeaconChain<T>>,
network_tx: &UnboundedSender<NetworkMessage<T::EthSpec>>,
validation_level: BroadcastValidation,
duplicate_status_code: StatusCode,
network_globals: Arc<NetworkGlobals<T::EthSpec>>,
) -> Result<Response, Rejection> {
let  seen_timestamp = timestamp_now();
let (unverified_block, unverified_blobs, is_locally_built_block) = match  proven
ProvenancedBlock::Local(block, blobs, _) => (block, blobs, true),
```

```
ProvenancedBlock::Builder(block, blobs, _) => (block, blobs, false),
};
// 블록 검증 및 처리 로직
// 네트워크로 블록 전파
// 응답 반환
}
```

요청 처리 과정

1. 필터 체인 구조

HTTP API는 Warp의 필터 체인을 사용하여 요청을 처리합니다:

```
// 버전 필터
let eth_v1 = single_version(V1);
let eth_v2 = single_version(V2);

// 체인 접근 필터
let chain_filter = warp::any()
.map(move || inner_ctx.chain.clone())
.and_then(|chain| async move {
match chain {
Some(chain) => Ok(chain),
None => Err(warp_utils::reject::custom_not_found(
```

```
"Beacon chain genesis has not yet been observed.".to_string(),
)),
}
});


// 동기화 상태 확인 필터
let  not_while_syncing_filter = warp::any()
.and(network_globals.clone())
.and(chain_filter.clone())
.then(move |network_globals: Arc<NetworkGlobals<T::EthSpec>>, chain: Arc<BeaconC
match *network_globals.sync_state.read() {
SyncState::SyncingFinalized { .. } => {
// 동기화 허용 범위 확인
}
SyncState::Synced => Ok(()),
// 기타 상태 처리
}
});
```

2. 작업 스케줄링

```
// TaskSpawner를 통한 비동기 작업 처리
let  task_spawner_filter = warp::any().map(move || TaskSpawner::new(beacon_proce
```

```
// 우선순위별 작업 실행
task_spawner.blocking_json_task(Priority::P1, move || {
// 실제 비즈니스 로직 실행
})
```

우선순위 시스템:

- `Priority::P0` : 검증자 클라이언트의 중요한 요청 (head 상태 조회 등)
- `Priority::P1` : 일반적인 API 요청
- `Priority::P2` : 낮은 우선순위 요청

3. 에러 처리

```
#[derive(Debug)]
pub  enum Error {
Warp(warp::Error),
Other(String),
}

// 통합 에러 처리
.recover(warp_utils::reject::handle_rejection)
```

에러 처리 전략:

- 표준 HTTP 상태 코드 사용

- 상세한 에러 메시지 제공
- 로깅을 통한 디버깅 지원

라우팅 구조

1. 엔드포인트 그룹화

```
// 최종 라우트 조합
let  routes = warp::get()
.and(
get_beacon_genesis
.uor(get_beacon_state_root)
.uor(get_beacon_state_fork)
.uor(get_beacon_state_finality_checkpoints)
.uor(get_beacon_state_validator_balances)
// ... 더 많은 GET 엔드포인트들
)
.boxed()
.uor(
warp::post().and(
// POST 엔드포인트들
post_beacon_blocks
.uor(post_beacon_blinded_blocks)
.uor(post_beacon_pool_attestations_v2)
```

```
// ... 더 많은 POST 엔드포인트들
)
)
.recover(warp_utils::reject::handle_rejection)
.with(tracing_logging())
.with(prometheus_metrics())
.with(cors_builder.build());
```

2. 미들웨어 체인

로깅 미들웨어:

```
pub fn tracing_logging() -> warp::filters::log::Log<impl Fn(warp::filters::log
warp::log::custom(move |info| {
let status = info.status();
let elapsed = info.elapsed().as_secs_f64() * 1000.0;
if status == StatusCode::OK || status == StatusCode::NOT_FOUND {
debug!(elapsed_ms = %elapsed, status = %status, "Processed HTTP API request");
} else {
warn!(elapsed_ms = %elapsed, status = %status, "Error processing HTTP API reques
}
})
}
```

메트릭 미들웨어:

```
pub fn prometheus_metrics() -> warp::filters::log::Log<impl Fn(warp::filters::
warp::log::custom(move |info| {
let path = categorize_path(info.path()); // 경로 분류
metrics::inc_counter_vec(&metrics::HTTP_API_PATHS_TOTAL, &[path]);
metrics::inc_counter_vec(&metrics::HTTP_API_STATUS_CODES_TOTAL, &[&info.status()
metrics::observe_timer_vec(&metrics::HTTP_API_PATHS_TIMES, &[path], info.elapsed
})
}
```

서버 시작 과정

1. 서버 구성

```
pub fn serve<T: BeaconChainTypes>(
ctx: Arc<Context<T>>,
shutdown: impl Future<Output = ()> + Send + Sync + 'static,
) -> Result<HttpServer, Error> {
let config = ctx.config.clone();
// CORS 설정
let cors_builder = warp::cors()
.allow_methods(vec!["GET", "POST"])
```

```
.allow_headers(vec!["Content-Type"]);

// 라우트 구성

let  routes = build_routes(ctx);

// 서버 시작

let  http_socket: SocketAddr = SocketAddr::new(config.listen_addr, config.listen

match  config.tls_config {

Some(tls_config) => {

// TLS 서버 시작

let (socket, server) = warp::serve(routes)

.tls()

.cert_path(tls_config.cert)

.key_path(tls_config.key)

.try_bind_with_graceful_shutdown(http_socket, shutdown)?;

Ok((socket, Box::pin(server)))

}

None => {

// 일반 HTTP 서버 시작

let (socket, server) = warp::serve(routes)

.try_bind_with_graceful_shutdown(http_socket, shutdown)?;

Ok((socket, Box::pin(server)))

}

}
```

```
}
```

2. 우아한 종료

```
// 종료 신호 처리
.try_bind_with_graceful_shutdown(http_socket, async {
shutdown.await;
})
```

서버는 종료 신호를 받으면 현재 처리 중인 요청을 완료한 후 안전하게 종료됩니다.

특수 기능

1. Server-Sent Events (SSE)

```
// GET /eth/v1/events
let  get_events = eth_v1
.and(warp::path("events"))
.and(warp::path::end())
.and(multi_key_query::<api_types::EventQuery>())
.then(|query_res: Result<api_types::EventQuery, warp::Rejection>| {
// SSE 스트림 생성 및 관리
});
```

지원하는 이벤트 타입:

- `head` : 새로운 헤드 블록
- `block` : 새로운 블록
- `attestation` : 새로운 증명
- `voluntary_exit` : 자발적 탈퇴
- `finalized_checkpoint` : 최종화된 체크포인트

2. 콘텐츠 타입 지원

```
// JSON과 SSZ 모두 지원
warp::header::exact(CONTENT_TYPE_HEADER, SSZ_CONTENT_TYPE_HEADER)
.and(post_beacon_blocks_ssz)
.uor(post_beacon_blocks) // JSON 버전
```

3. 버전 관리

```
// API 버전별 엔드포인트 분리
let  eth_v1 = single_version(V1);
let  eth_v2 = single_version(V2);
let  eth_v3 = single_version(V3);
```

성능 최적화

1. 비동기 처리

모든 요청은 `TaskSpawner` 를 통해 비동기적으로 처리되어 블로킹을 방지합니다.

2. 캐싱 전략

- 검증자 듀티는 캐시되어 반복 요청 시 빠른 응답 제공
- 상태 정보는 필요시에만 계산

3. 메모리 관리

- 대용량 응답은 스트리밍으로 처리
- 시스템 정보는 주기적으로 갱신

보안 고려사항

1. CORS 설정

```
let cors_builder = warp_utils::cors::set_builder_origins(
builder,
config.allow_origin.as_deref(),
(config.listen_addr, config.listen_port),
)?;
```

2. TLS 지원

```
#[derive(PartialEq, Debug, Clone, Serialize, Deserialize)]
pub struct TlsConfig {
pub cert: PathBuf,
pub key: PathBuf,
```

```
}
```

3. 요청 검증

모든 입력은 타입 안전성과 유효성 검사를 거칩니다.

모니터링 및 로깅

1. 메트릭 수집

- 요청 수 및 응답 시간
- 상태 코드별 통계
- 엔드포인트별 사용량

2. 구조화된 로깅

```
info!(
listen_address = %http_server.0,
"HTTP API started"
);
```

3. 헬스 체크

```
// GET /eth/v1/node/health
// 노드 상태 확인 엔드포인트
```

결론

Lighthouse의 HTTP API 서버는 Ethereum Beacon API 표준을 완전히 구현하면서도 성능과 안정성을 고려한 설계를 보여줍니다. Warp 프레임워크의 필터 체인을 활용한 모듈화된 구조, 비동기 처리를 통한 성능 최적화, 그리고 포괄적인 에러 처리와 모니터링 기능을 제공합니다. 이를 통해 검증자 클라이언트와 외부 도구들이 beacon node와 효율적으로 상호작용할 수 있는 견고한 인터페이스를 제공합니다.